

```
scripts/export_repo_to_pdf.py
```

```
import os
from pathlib import Path
from typing import Iterable, List

REPO_ROOT = Path(__file__).resolve().parents[1]
OUTPUT_DIR = REPO_ROOT / "docs" / "code_pdfs"

# File extensions considered text for export
TEXT_EXTENSIONS = {
    ".py",
    ".md",
    ".txt",
    ".json",
    ".sql",
    ".env",
}

EXCLUDE_DIRS = {
    ".git",
    "__pycache__",
    "docs/code_pdfs",
}

MAX_LINES_PER_PAGE = 55
TOP_MARGIN = 760
LINE_HEIGHT = 12

def iter_text_files(root: Path) -> Iterable[Path]:
    for dirpath, dirnames, filenames in os.walk(root):
        rel_dir = Path(dirpath).relative_to(root)
        if any(part in EXCLUDE_DIRS for part in rel_dir.parts):
            dirnames[:] = []
            continue
        dirnames[:] = [d for d in dirnames if d not in EXCLUDE_DIRS]
        for filename in filenames:
            path = Path(dirpath) / filename
            if path.suffix.lower() in TEXT_EXTENSIONS:
                yield path

def escape_text(text: str) -> str:
    return text.replace("\\", "\\").replace("(", "\\(").replace(")", "\\)")

def chunk_lines(lines: List[str], size: int) -> List[List[str]]:
    return [lines[i : i + size] for i in range(0, len(lines), size)] or [[]]

def build_content_stream(lines: List[str], title: str) -> str:
    parts = [
        "BT",
        "/F1 12 Tf",
```

```
scripts/export_report_to_pdf.py
```

```
    f"({escape_text(title)}) Tj",
    "T*",
    "/F1 9 Tf",
    f"{LINE_HEIGHT} TL",
]
for line in lines:
    parts.append(f"({escape_text(line)}) Tj")
    parts.append("T*")
parts.append("ET")
return "\n".join(parts)
```

```
def build_pdf(pages: List[List[str]], title: str) -> bytes:
    objects = []
```

```
    # Font object
```

```
    font_obj_num = len(objects) + 1
```

```
    objects.append("<< /Type /Font /Subtype /Type1 /BaseFont /Courier >>")
```

```
    page_objs = []
```

```
    content_objs = []
```

```
    for page_lines in pages:
```

```
        content = build_content_stream(page_lines, title)
```

```
        content_bytes = content.encode("utf-8")
```

```
        content_obj_num = len(objects) + 1
```

```
        content_objs.append(content_obj_num)
```

```
        objects.append(f"<< /Length {len(content_bytes)} >>\nstream\n{content}\nendstream")
```

```
        page_obj_num = len(objects) + 1
```

```
        page_objs.append(page_obj_num)
```

```
        page_obj = (
```

```
            "<< /Type /Page /Parent 0 0 R /MediaBox [0 0 612 792] "
```

```
            f"/Contents {content_obj_num} 0 R /Resources << /Font << /F1 {font_obj_num} 0 R >> >> >>
```

```
        )
```

```
        objects.append(page_obj)
```

```
    # Pages object
```

```
    pages_obj_num = len(objects) + 1
```

```
    kids = " ".join(f"{num} 0 R" for num in page_objs)
```

```
    objects.append(f"<< /Type /Pages /Kids [{kids}] /Count {len(page_objs)} >>")
```

```
    # Replace parent references now that pages object number is known
```

```
    for idx, page_obj_num in enumerate(page_objs):
```

```
        page_obj = objects[page_obj_num - 1]
```

```
        objects[page_obj_num - 1] = page_obj.replace("0 0 R", f"{pages_obj_num} 0 R", 1)
```

```
    # Catalog object
```

```
    catalog_obj_num = len(objects) + 1
```

```
    objects.append(f"<< /Type /Catalog /Pages {pages_obj_num} 0 R >>")
```

```
    # Build PDF body
```

```
    offsets = []
```

```
    pdf_parts = ["%PDF-1.4\n"]
```

```
    for obj_num, content in enumerate(objects, start=1):
```

```
scripts/export_repo_to_pdf.py
if __name__ == '__main__':
    build_pdf(

```

```
        pdf_parts.append(f"obj_{obj_num} 0 obj\n{{content}}\nendobj\n")

    xref_offset = sum(len(p.encode("utf-8")) for p in pdf_parts)
    pdf_parts.append("xref\n")
    pdf_parts.append(f"0 {len(objects) + 1}\n")
    pdf_parts.append("0000000000 65535 f \n")
    for offset in offsets:
        pdf_parts.append(f"{{offset:010d}} 00000 n \n")

    pdf_parts.append(
        "trailer\n"
        f"<< /Size {len(objects) + 1} /Root {{catalog_obj_num}} 0 R >>\n"
        f"startxref\n{{xref_offset}}\n%%EOF"
    )
    return b"".join(part.encode("utf-8") for part in pdf_parts)
```

```
def add_file_to_pdf(file_path: Path) -> Path:
```

```
    try:
        content = file_path.read_text(encoding="utf-8")
    except UnicodeDecodeError:
        content = file_path.read_text(encoding="utf-8", errors="replace")
    lines = [f"Path: {{file_path.relative_to(REPO_ROOT)}}"] + content.splitlines()
    page_chunks = chunk_lines(lines, MAX_LINES_PER_PAGE)
    pdf_bytes = build_pdf(page_chunks, title=str(file_path.relative_to(REPO_ROOT)))
    output_path = OUTPUT_DIR / f"{{file_path.relative_to(REPO_ROOT).as_posix().replace('/', '_')}}.pdf"
    output_path.parent.mkdir(parents=True, exist_ok=True)
    output_path.write_bytes(pdf_bytes)
    return output_path
```

```
def main():
```

```
    OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
    for file_path in iter_text_files(REPO_ROOT):
        output = add_file_to_pdf(file_path)
        print(f"Generated {{output}}")
```

```
if __name__ == "__main__":
```

```
    main()
```